# Course Management: Edit and Delete

React UI + Axios + Existing Spring Boot REST API

**Step-by-step laboratory guide** covering API service functions, Edit and Delete buttons, form state, event handlers, the complete working component, and the fix for `course.code.trim is not a function`.

| | |
|---|---|
| Frontend | React, JavaScript, Bootstrap, Axios |
| Existing API | http://localhost:8080/api/v1/courses |
| UI location | src/components/Courses.jsx |
| API location | src/services/courseApi.js |
| Scope | Display, edit and delete courses; no backend rewrite |

**Assumption:** The backend supports GET /api/v1/courses, PUT /api/v1/courses/{id}, and DELETE /api/v1/courses/{id}. Verify the exact mappings and JSON field types in your controller before testing.

# 1. Learning objectives and architecture

The existing GET list remains in place. Students will add an Actions column, an edit form, a confirmation dialog, and calls to the existing REST endpoints.

```
React Courses.jsx
    |-- GET    /api/v1/courses       --> display table
    |-- PUT    /api/v1/courses/{id}  --> save edit
    `-- DELETE /api/v1/courses/{id}  --> remove row
                  |
                  v
             Spring Boot API
```

# 2. Project structure

```
src/
|-- components/
|   `-- Courses.jsx
|-- services/
|   `-- courseApi.js
|-- App.jsx
`-- main.jsx
```

Keep the Courses component inside **components**, matching the project built during class. The service file holds HTTP calls, while the component controls presentation and user interaction.

# 3. Add update and delete API functions

## src/services/courseApi.js

```
import axios from "axios";

const API_URL = "http://localhost:8080/api/v1/courses";

export const getCourses = () => {
    return axios.get(API_URL);
};

export const updateCourse = (id, course) => {
    return axios.put(`${API_URL}/${id}`, course);
};

export const deleteCourse = (id) => {
    return axios.delete(`${API_URL}/${id}`);
};
```

| | |
|---|---|
| import axios | Makes the Axios HTTP client available in this module. |
| const API_URL | Stores the collection URL in one constant so requests use the same backend address. |
| export const getCourses | Exports a function for retrieving the course list. |
| axios.get(API_URL) | Sends HTTP GET and returns a Promise for the server response. |
| updateCourse(id, course) | Accepts the ID of the existing record and the edited object. |
| `${API_URL}/${id}` | Uses a template literal to append an ID, for example /courses/101. |
| axios.put(url, course) | Sends HTTP PUT with the edited object serialized as JSON. |

| | |
|---|---|
| import axios | Makes the Axios HTTP client available in this module. |
| deleteCourse(id) | Accepts the ID of the course selected for removal. |
| axios.delete(url) | Sends HTTP DELETE to the resource URL. |

## 4. Add the Actions column

```
<thead className="table-dark">
    <tr>
        <th>ID</th>
        <th>Code</th>
        <th>Title</th>
        <th>Credits</th>
        <th>Actions</th>
    </tr>
</thead>

<tbody>
    {courses.map((item) => (
        <tr key={item.id}>
            <td>{item.id}</td>
            <td>{item.code}</td>
            <td>{item.title}</td>
            <td>{item.credits}</td>
            <td>
                <button className="btn btn-warning btn-sm me-2"
                    onClick={() => handleEdit(item)}>Edit</button>
                <button className="btn btn-danger btn-sm"
                    onClick={() => handleDelete(item.id)}>Delete</button>
            </td>
        </tr>
    ))}
</tbody>
```

| | |
|---|---|
| <th>Actions</th> | Adds the heading for the new fifth column. |
| courses.map(...) | Transforms each course object into one table row. |
| key={item.id} | Gives React a stable key to reconcile list items. |
| className | Applies Bootstrap classes; btn-warning is yellow and btn-danger is red. |
| me-2 | Adds a small margin after the Edit button. |
| onClick={() => handleEdit(item)} | Passes the entire selected course object to the Edit handler only when clicked. |
| onClick={() => handleDelete(item.id)} | Passes the selected course ID to the Delete handler only when clicked. |

## 5. Understand the editing state

```
const [courses, setCourses] = useState([]);
const [editingId, setEditingId] = useState(null);
const [course, setCourse] = useState({
    code: "",
    title: "",
    credits: ""
});
const [message, setMessage] = useState("");
const [error, setError] = useState("");
```

| | |
|---|---|
| courses / setCourses | Stores the current list returned by Spring Boot. |
| editingId / setEditingId | null means no edit form; a number or ID means the user is editing that record. |
| course / setCourse | Stores the controlled input values in the edit form. |
| message / setMessage | Displays success feedback after a successful operation. |
| error / setError | Displays a readable error when loading or saving fails. |

## 6. Fix the JavaScript trim() error

The reported runtime error is **TypeError: course.code.trim is not a function**. It occurs because the value is not a string. For example, if the backend returns `code: 401`, the number 401 has no string method named `trim()`.

```
// Incorrect when course.code is a number:
code: course.code.trim(),

// Safe conversion before trimming:
code: String(course.code ?? "").trim(),
title: String(course.title ?? "").trim(),
credits: Number(course.credits)
```

The **??** operator substitutes an empty string only when the original value is null or undefined. **String(...)** produces a string, and **trim()** removes leading/trailing whitespace. **Number(...)** converts the credits input into a numeric value.

**Important:** A code such as IS-401 should normally remain a string in the backend. Conversion in React avoids the JavaScript error but cannot recover a prefix already lost in the API response. Check the Spring Boot model and request DTO if the API uses a numeric code field.

## 7. Complete Courses.jsx (ready to paste)

Replace the current Courses component with the following code. The GET effect loads data, Edit fills the form, Update calls PUT, and Delete asks for confirmation before calling DELETE.

### src/components/Courses.jsx

```
import { useEffect, useState } from "react";
import { getCourses, updateCourse, deleteCourse }
    from "../services/courseApi";

function Courses() {
    const [courses, setCourses] = useState([]);
    const [editingId, setEditingId] = useState(null);
    const [course, setCourse] = useState({
        code: "", title: "", credits: ""
    });
    const [message, setMessage] = useState("");
    const [error, setError] = useState("");

    useEffect(() => {
        let active = true;
        const fetchCourses = async () => {
            try {
                const response = await getCourses();
                if (active) setCourses(response.data);
            } catch (err) {
                if (active) setError("Failed to load courses.");
                console.error(err);
            }
        };
        fetchCourses();
        return () => { active = false; };
    }, []);

    const loadCourses = async () => {
        const response = await getCourses();
        setCourses(response.data);
    };

    const handleEdit = (selectedCourse) => {
        setEditingId(selectedCourse.id);
        setCourse({
            code: String(selectedCourse.code ?? ""),
            title: String(selectedCourse.title ?? ""),
            credits: String(selectedCourse.credits ?? "")
        });
        setMessage("");
        setError("");
    };

    const handleChange = (event) => {
```

```
        const { name, value } = event.target;
        setCourse(previous => ({ ...previous, [name]: value }));
    };

    const handleUpdate = async (event) => {
        event.preventDefault();
        const updatedCourse = {
            code: String(course.code ?? "").trim(),
            title: String(course.title ?? "").trim(),
            credits: Number(course.credits)
        };
        try {
            await updateCourse(editingId, updatedCourse);
            await loadCourses();
            setEditingId(null);
            setCourse({ code: "", title: "", credits: "" });
            setError("");
            setMessage("Course updated successfully!");
        } catch (err) {
            console.error(err);
            setError("Failed to update course. Check the API response.");
        }
    };

    const handleDelete = async (id) => {
        if (!window.confirm("Delete this course?")) return;
        try {
            await deleteCourse(id);
            await loadCourses();
            if (editingId === id) setEditingId(null);
            setError("");
            setMessage("Course deleted successfully!");
        } catch (err) {
            console.error(err);
            setError("Failed to delete course. Check the API response.");
        }
    };

    return (
        <div className="container mt-4">
            <h2 className="text-center mb-4">Course Management</h2>
            {message && <div className="alert alert-success">{message}</div>}
            {error && <div className="alert alert-danger">{error}</div>}

            {editingId !== null && (
```

```
            <div className="card mb-4 shadow-sm">
                <div className="card-header bg-warning">
                    <strong>Edit Course</strong>
                </div>
                <div className="card-body">
                    <form onSubmit={handleUpdate}>
                        <div className="row g-3">
                            <div className="col-md-3">
                                <label className="form-label">Course Code</label>
                                <input type="text" name="code"
                                    className="form-control"
                                    value={course.code}
                                    onChange={handleChange} required />
                            </div>
                            <div className="col-md-5">
                                <label className="form-label">Course Title</label>
                                <input type="text" name="title"
                                    className="form-control"
                                    value={course.title}
                                    onChange={handleChange} required />
                            </div>
                            <div className="col-md-2">
                                <label className="form-label">Credits</label>
                                <input type="number" name="credits" min="1"
                                    className="form-control"
                                    value={course.credits}
                                    onChange={handleChange} required />
                            </div>
                            <div className="col-md-2 d-flex align-items-end">
                                <button type="submit"
                                    className="btn btn-success w-100">
                                    Update
                                </button>
                            </div>
                        </div>
                        <button type="button"
                            className="btn btn-secondary mt-3"
                            onClick={() => setEditingId(null)}>
                            Cancel
                        </button>
                    </form>
                </div>
            </div>
        )}
```

```
            <div className="table-responsive">
                <table className="table table-bordered table-hover text-center">
                    <thead className="table-dark">
                        <tr>
                            <th>ID</th><th>Code</th><th>Title</th>
                            <th>Credits</th><th>Actions</th>
                        </tr>
                    </thead>
                    <tbody>
                        {courses.map((item) => (
                            <tr key={item.id}>
                                <td>{item.id}</td>
                                <td>{item.code}</td>
                                <td>{item.title}</td>
                                <td>{item.credits}</td>
                                <td>
                                    <button type="button"
                                        className="btn btn-warning btn-sm me-2"
                                        onClick={() => handleEdit(item)}>
                                        Edit
                                    </button>
                                    <button type="button"
                                        className="btn btn-danger btn-sm"
                                        onClick={() => handleDelete(item.id)}>
                                        Delete
                                    </button>
                                </td>
                            </tr>
                        ))}
                    </tbody>
                </table>
            </div>
        </div>
    );
}

export default Courses;
```

## 8. Line-by-line explanation: imports, state and GET

| | |
|---|---|
| import { useEffect, useState } | Imports React Hooks for state management and the initial API fetch. |
| import { getCourses, ... } | Imports the existing GET and newly added PUT/DELETE service functions. |
| function Courses() | Defines the React function component that represents the course page. |
| useState([]) | Initializes the course list as an empty array before the API response. |
| useState(null) | Initializes editingId to null so the edit form is initially hidden. |
| useState({ ... }) | Initializes form values with empty strings. |
| useEffect(() => {...}, []) | Runs the GET synchronization when the component mounts. |
| let active = true | Tracks whether this particular effect instance is still active. |
| const fetchCourses = async () => | Declares an asynchronous function inside the effect. |
| await getCourses() | Waits for the GET request to resolve. |
| if (active) setCourses(response.data) | Updates state only while the effect instance is active. |
| catch (err) | Handles a failed request and logs the technical error. |

| | |
|---|---|
| import { useEffect, useState } | Imports React Hooks for state management and the initial API fetch. |
| fetchCourses() | Starts the asynchronous function. |
| return () => { active = false; } | Cleanup prevents an outdated request from changing state after the effect is cleaned up. |
| loadCourses | A reusable function that refreshes the list after a successful mutation. |

## 9. Line-by-line explanation: editing and form inputs

| | |
|---|---|
| handleEdit(selectedCourse) | Runs when a user clicks Edit for a particular row. |
| setEditingId(selectedCourse.id) | Stores the selected course ID, causing the conditional form to appear. |
| String(selectedCourse.code ?? "") | Normalizes a numeric, null or string code into a string suitable for the input. |
| String(selectedCourse.title ?? "") | Normalizes the title to a string. |
| String(selectedCourse.credits ?? "") | Stores the credits input as a string, matching HTML input behavior. |
| setMessage(""); setError("") | Clears previous feedback when starting another edit. |
| handleChange(event) | Handles typing in any of the three form inputs. |
| event.target | Refers to the input that generated the change event. |
| const { name, value } | Extracts the input name (code/title/credits) and its current value. |
| setCourse(previous => ... ) | Uses a functional state update to avoid stale state. |
| ...previous | Preserves the properties that were not changed. |
| [name]: value | Updates only the property matching the input name. |

## 10. Line-by-line explanation: update

| | |
|---|---|
| handleUpdate = async (event) | Defines the handler invoked by form submission. |
| event.preventDefault() | Prevents the browser from reloading the page on submit. |
| const updatedCourse = {...} | Creates the JSON-ready object to send to the API. |
| String(course.code ?? "").trim() | Converts the code to a string and removes surrounding spaces; fixes the reported trim error. |
| String(course.title ?? "").trim() | Converts the title to a string and removes surrounding spaces. |
| Number(course.credits) | Converts the numeric input string into a number for the backend. |
| await updateCourse(editingId, updatedCourse) | Sends PUT to /courses/{id} and waits for the server response. |
| await loadCourses() | Refreshes the table with the latest backend data. |

| | |
|---|---|
| handleUpdate = async (event) | Defines the handler invoked by form submission. |
| setEditingId(null) | Closes the edit form. |
| setCourse({ ... }) | Clears the form values. |
| setMessage(...) | Shows a success message. |
| catch (err) | Handles a failed PUT or refresh and shows an error. |

## 11. Line-by-line explanation: delete and JSX

| | |
|---|---|
| handleDelete = async (id) | Defines the handler for a row-specific delete action. |
| window.confirm(...) | Shows a native browser confirmation dialog. |
| if (!confirm) return | Stops immediately if the user cancels. |
| await deleteCourse(id) | Sends DELETE to /courses/{id}. |
| await loadCourses() | Refreshes the visible course list after the deletion. |
| editingId === id | Checks whether the deleted course is currently open for editing. |
| setEditingId(null) | Closes the form if it was editing the deleted record. |
| return (...) | Returns the JSX describing the UI. |
| {message && ...} | Conditionally shows a Bootstrap success alert. |
| {error && ...} | Conditionally shows a Bootstrap error alert. |
| {editingId !== null && (...)} | Displays the edit form only when a course has been selected. |
| <form onSubmit={handleUpdate}> | Runs the update handler when the user submits the form. |
| value={course.code} | Makes the input controlled by React state. |
| onChange={handleChange} | Updates React state when the user types. |
| type="number" min="1" | Uses numeric input with a minimum of 1; backend validation is still required. |
| type="button" | Prevents Edit, Delete and Cancel from submitting a form. |
| courses.map((item) => ... ) | Builds one table row for each course returned by the API. |
| onClick={() => handleEdit(item)} | Invokes Edit with the selected course object. |
| onClick={() => handleDelete(item.id)} | Invokes Delete with the selected course ID. |
| export default Courses | Allows App.jsx to import this component. |

## 12. Example API requests

```
PUT http://localhost:8080/api/v1/courses/101
Content-Type: application/json

{
  "code": "IS-401",
  "title": "Advanced Enterprise Web Development",
  "credits": 3
}

DELETE http://localhost:8080/api/v1/courses/103
```

A successful DELETE may return **204 No Content**. Axios resolves successfully even though there is no response body. A PUT may return an updated object or another successful status, depending on your existing controller.

## 13. Troubleshooting and validation

| | |
|---|---|
| trim is not a function | Inspect typeof selectedCourse.code. Normalize values with String(...).trim() and verify the backend DTO. |
| 400 Bad Request | Check the exact backend request schema and validation rules; inspect Network > Payload. |
| 404 Not Found | Check the ID and PUT/DELETE endpoint mappings. |
| 405 Method Not Allowed | Verify the HTTP verb; PUT and DELETE cannot be tested by simply opening a URL in the address bar. |
| CORS blocked | Verify the existing Spring Boot CORS configuration allows http://localhost:5173 and PUT/DELETE/OPTIONS. |
| Table does not refresh | Inspect the GET request after mutation and check response.data is an array. |
| Delete fails | Check backend constraints: a course may be referenced by sections or registrations. |
| React duplicate GET in development | React StrictMode may run an effect setup twice in development; this is expected and does not imply two user clicks. |

## 14. Practical student testing checklist

1. Run the Spring Boot backend and the Vite frontend. 2. Open **http://localhost:5173/courses**. 3. Verify all courses appear. 4. Click Edit on course 101 and verify its existing values populate the form. 5. Change the title and submit Update. 6. Refresh the page and confirm the change persists. 7. Click Delete on another test course. 8. Test Cancel and Confirm separately. 9. Inspect the Network tab for the correct PUT/DELETE methods and response codes.

## 15. References


React documentation: https://react.dev/learn

React effects: https://react.dev/reference/react/useEffect

React state: https://react.dev/reference/react/useState

Axios API: https://axios-http.com/docs/api_intro

Bootstrap forms: https://getbootstrap.com/docs/5.3/forms/overview/

Spring CORS: Spring Framework MVC CORS documentation